Збір Даних

Методи польових досліджень програмного забезпечення

При проведенні польових досліджень важливо отримати точну та достовірну інформацію про досліджуване явище. Інтерв'ю та анкети є найбільш простими інструментами, але дані, які вони виробляють, зазвичай представляють неповну картину. Наприклад, припустимо, що ваша мета - оцінити, які функції мови програмування найбільш схильні до помилок. Розробник може дати вам загальні думки та анекдотичні докази з цього приводу; однак ви отримаєте набагато точнішу інформацію, записуючи та аналізуючи практику роботи розробника, їх зусилля щодо багаторазового редагування та компіляції коду. Для цього типу досліджень використовуються такі методи, як протоколи мислення вголос та робочі щоденники (think-aloud protocols and work diaries). Щоб дізнатися про різні аспекти явища, часто найкраще використовувати кілька методів збору даних (три і більше). Потім аналізуються отримані дані, щоб триангулювати практику роботи.

Table 1. Data collection techniques suitable for field studies of software engineering.

Category	Technique
First Degree (direct involvement of software engineers)	Inquisitive techniques • Brainstorming and Focus Groups • Interviews • Questionnaires • Conceptual Modeling Observational techniques • Work Diaries • Think-aloud Protocols • Shadowing and Observation Synchronized Shadowing • Participant Observation (Joining the Team)
Second Degree (indirect involvement of software engineers)	• Instrumenting Systems • Fly on the Wall (Participants Taping Their Work)
Third Degree (study of work artifacts only)	• Analysis of Electronic Databases of Work Performed • Analysis of Tool Use Logs • Documentation Analysis • Static and Dynamic Analysis of a System

Контакт першого ступеня вимагає прямого доступу до популяції учасників. Контакт другого ступеня вимагає доступу до середовища учасників під час роботи, але без необхідності прямого доступу до учасників або взаємодії учасників та дослідників. Нарешті, контакт третього ступеня вимагає доступу лише до робочих артефактів, таких як вихідний код або документація.

Ця таксономія є інформативною, оскільки ступінь контакту відображає види даних, які можуть бути зібрані, необхідні ресурси, гнучкість, доступну досліднику, та надійність отриманої інформації. Тісний контакт з суб'єктами вимагатиме міцніших робочих відносин, ніж ненав'язливі архівні дослідження. Враховуючи це, методи нижчого ступеня слід вибирати лише тоді, коли вони є єдиним способом отримання даних, необхідних для відповіді на дослідницьке питання, тобто коли вам потрібно знати, про що думають інженери.

Всі методи, від найпростіших до найскладніших, потрібно застосовувати з обережністю та повагою. Питання, спостережні сесії та інші заходи повинні бути ретельно сплановані, щоб переконатися, що зібрані дані мають сенс. Погано сформульоване запитання призводить до неоднозначних відповідей, які неможливо інтерпретувати чи проаналізувати. Крім того, до учасників слід ставитися з повагою, тому що вони, перш за все, люди в соціальних (виробничих) ситуаціях. Дослідники повинні бути чутливими до збоїв, які може спричинити присутність дослідника.

Вибір методу збору даних

Вибір методу збору даних повинен проводитися в контексті дослідницької мети та питання. Пропонується, щоб метод дослідження був обраний так само, як метрики в інженерії програмного забезпечення, тобто дотримуючись методу, аналогічного V. Basili GQM підхіду.

Першим кроком у розробці будь-якого дослідження є встановлення набору добре зрозумілих цілей, оскільки наступні рішення залежать від них. Цілі повинні описувати явище, що вивчається, і мету дослідження, тобто як результати буде використано. Польові дослідження мають різні цілі. Наприклад, розробити вимоги до інструментів, вивчаючи інженерів-програмістів, або краще зрозуміти, як інженери-програмісти виконують певну діяльність, наприклад, таку як супроводження, або вийти за рамки простого розуміння і отримати рекомендації щодо вдосконалення процесів.

Цілі дослідження керують формулюванням дослідницьких питань, які, у свою чергу, керують проектуванням дослідження, який, у свою чергу, диктує вибір методу (методів) збору даних. Припускаємо, що при виборі методу слід враховувати три фактори: ступінь доступу, до інженерів-програмістів, обсяг вироблених даних та тип дослідницьких питань.

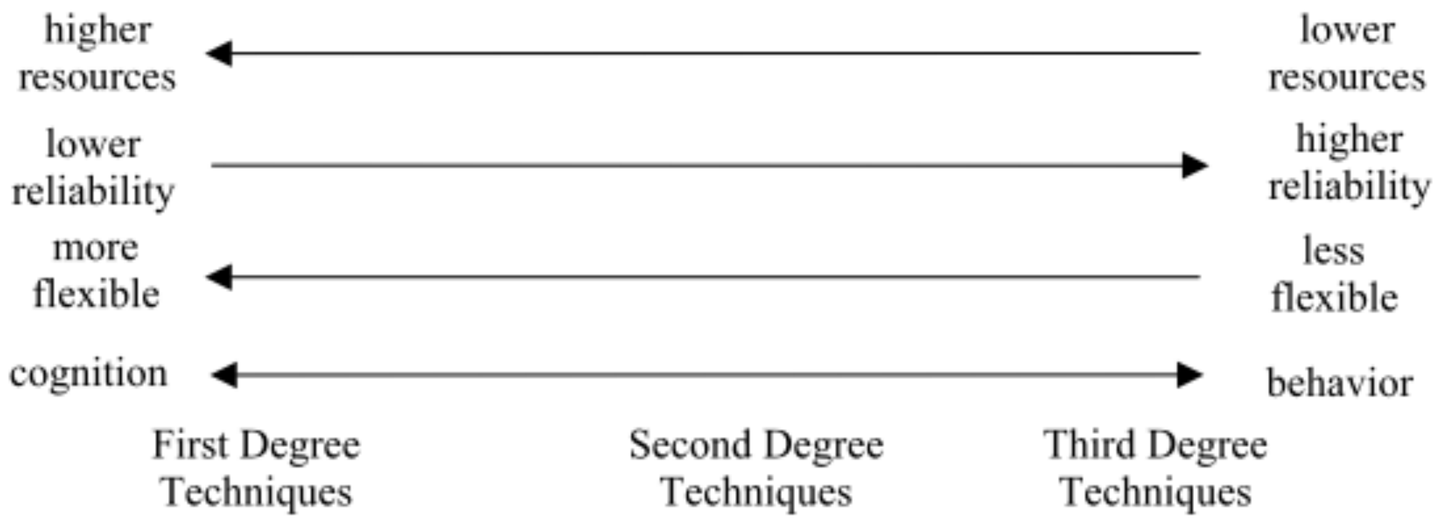
У наступної таблиці 2 представлено методи збору даних, де у другій колонці показано типи питань, на які можна отримати відповіді. Дослідник може вибрати набір методів, які разом дадуть відповідь на необхідні запитання. Третя колонка в таблиці, вказує на кількість даних. Слід зазначити, що ступінь контакту не безпосередньо пов'язано з кількістю зібраних даних.

Technique	Used by researchers when their goal is to understand:	Volume of data	Also used by software engineers for:
First Order Techniques			
Brainstorming and Focus Groups	Ideas and general background about the process and product, general opinions (also useful to enhance participant rapport)	Small	Requirements gathering, project planning
Surveys	General information (including opinions) about process, product, personal knowledge etc.	Small to Large	Requirements and evaluation
Conceptual modeling	Mental models of product or process	Small	Requirements
Work Diaries	Time spent or frequency of certain tasks (rough approximation, over days or weeks)	Medium	
Think-aloud sessions	Mental models, goals, rationale and patterns of activities	Medium to large	UI evaluation
Shadowing and Observation	Time spent or frequency of tasks (intermittent over relatively short periods), patterns of activities, some goals and rationale	Small	Advanced approaches to use case or task analysis
Participant observation (joining the team)	Deep understanding, goals and rationale for actions, time spent or frequency over a long period	Medium	
Second Order Techniques			
Instrumenting systems	Software usage over a long period, for many participants	Large	Software usage analysis
Fly in the wall	Time spent intermittently in one location, patterns of activities (particularly collaboration)	Medium	
Third Order Techniques			
Analysis of work databases	Long-term patterns relating to software evolution, faults etc.	Large	Metrics gathering
Analysis of tool use logs	Details of tool usage	Large	
Documentation analysis	Design and documentation practices, general understanding	Medium	Reverse engineering
Static and dynamic analysis	Design and programming practices, general understanding	Large	Program comprehension, metrics, testing, etc.

Ці три виміру, ступінь контакту, тип дослідницького питання та обсяг даних, взятих разом, припускають, який тип дослідження буде найбільш підходящим. Дослідження можуть бути виконані там, де питання широкі, знань мало та мало даних для порівняльного аналізу. Дослідження, які зосереджено на аналізі даних підходить для кількісної оцінки явища, наприклад, скільки програмістів є і які мови вони знають. Багато польових досліджень у програмному забезпеченні, як правило, має дослідницький характер, тому що ми ще збираємо базові знання про людські фактори, що оточують розробку та обслуговування програмного забезпечення. Як результат зазвичай тематичне дослідження призводить до теорії або моделі, які можна перевірити пізніше. У міру зростання нашої бази знань ми можемо використовувати проекти, які тестують ці теорії або моделі.

Компромиси між методами збору даних

Перш ніж розглядати окремі методи надамо деякі загальні рекомендації щодо вибору серед них. Три категорії методів збору даних різняться з точки зору вартості ресурсів, необхідних для збору даних, їх надійності, гнучкості та явища. Відносні переваги різних категорій показані на рис.



Вартість - це функція зусиль, необхідних для збору даних, використовуваної техніки ведення обліку, кількості вироблених даних та зусиль, необхідних для аналізу даних. Загалом, методи нижчого ступеня дорожчі у використанні, оскільки вони вимагають більше часу та зусиль від дослідників та учасників дослідження. Методи, які виробляють більше даних, вимагають більше часу для аналізу цих даних. Комп'ютерні записи легше аналізувати, оскільки програмне забезпечення може бути залучене до даних, більше, ніж інші джерела даних, такі як відеокасети. Деякі винятки можуть виникнути при роботі з дуже великими журналами коду або інструментів.

Люди, як правило, не є надійними репортерами, оскільки вони часто не згадують минулі події з високим ступенем точності. Записи про діяльність, такі як стрічки, робочі продукти та сховища, як правило, більш надійні. Однак необхідно дотримуватися обережності при інтерпретації цих джерел даних, оскільки вони можуть бути несумісними, внутрішньо або один з одним.

Незважаючи на свої недоліки, методи першого ступеня безцінні через їх гнучкість і явище, для вивчення якого вони можуть бути використані. Існуючі журнали та сховища прості у використанні, але наявні дані дуже обмежені. Інженерів, з іншого боку, можна запитати про набагато ширший спектр тем. Методи другого ступеня лежать десь посередині. Однак всі ці методи все ще можуть бути використані для дослідницьких питань.

Нарешті, необхідний певний контакт з інженерами, щоб з'ясувати, що вони думають або відчувають. Більш віддалені методи можуть лише розповісти вам, що вони робили, однак це не проблема, якщо дії або практика роботи є основним інтересом дослідження. Деякі висновки можуть бути зроблені щодо пізнання з поведінки, але вони повинні бути підтверджені прямими запитами.

Підсумовуючи, кожна категорія методів має свої недоліки, тому необхідно використовувати відповідну комбінацію для збору даних, необхідних для надання доказів та відповіді на дослідницькі питання. Методи першого та другого ступеня є більш складними, ніж методи третього ступеня, але в багатьох ситуаціях це аргумент на їхню користь.

Огляд методів збору даних

Методи першого ступеня: пряме залучення інженерів

Перші п'ять методів, перелічених у таблиці, - це те, що називають допитливими методами, тоді як інші - в першу чергу спостережні. Кожен тип підходить для збору різних типів інформації від інженерів. Допитливі методи першого ступеня дозволяють експериментатору отримати загальне розуміння процесу розробки програмного забезпечення. Такі методи, мабуть, є єдиним способом оцінити, наскільки приємними або мотивуючими є використання певних інструментів або виконання певних видів діяльності. Однак вони часто суб'єктивні, і, крім того, не дозволяють проводити точні вимірювання часу.

Technique	Used by researchers when their goal is to understand:	Volume of data	Also used by software engineers for:
First Order Techniques			
Brainstorming and Focus Groups	Ideas and general background about the process and product, general opinions (also useful to enhance participant rapport)	Small	Requirements gathering, project planning
Surveys	General information (including opinions) about process, product, personal knowledge etc.	Small to Large	Requirements and evaluation
Conceptual modeling	Mental models of product or process	Small	Requirements
Work Diaries	Time spent or frequency of certain tasks (rough approximation, over days or weeks)	Medium	
Think-aloud sessions	Mental models, goals, rationale and patterns of activities	Medium to large	UI evaluation
Shadowing and Observation	Time spent or frequency of tasks (intermittent over relatively short periods), patterns of activities, some goals and rationale	Small	Advanced approaches to use case or task analysis
Participant observation (joining the team)	Deep understanding, goals and rationale for actions, time spent or frequency over a long period	Medium	
Second Order Techniques			
Instrumenting systems	Software usage over a long period, for many participants	Large	Software usage analysis
Fly in the wall	Time spent intermittently in one location, patterns of activities (particularly collaboration)	Medium	
Third Order Techniques			
Analysis of work databases	Long-term patterns relating to software evolution, faults etc.	Large	Metrics gathering
Analysis of tool use logs	Details of tool usage	Large	
Documentation analysis	Design and documentation practices, general understanding	Medium	Reverse engineering
Static and dynamic analysis	Design and programming practices, general understanding	Large	Program comprehension, metrics, testing, etc.

Методи спостереження першого ступеня забезпечують зображення досліджуваних явищ у реальному часі. Однак важче аналізувати дані, як тому, що вони щільні, так і тому, що вони вимагають значних знань для правильної інтерпретації. Методи спостереження можуть використовуватися у випадково обраний час або коли інженер займається певним видом діяльності (наприклад, щоразу, коли використовує відладчик). Методи спостереження завжди ризикують змінити процес, просто спостерігаючи за ним; ефект Хоторна був вперше виявлений, коли група дослідників виявила, що результат не пов'язаний з умовами навколишнього середовища, як очікувалося, а скоріше з тим, чи спостерігалися працівники. Тому ретельний розгляд цього ефекту виправданий при проведенні дослідження та поясненні його мети та протоколу учасникам дослідження.

Мозковий штурм та фокус-групи.

У мозковому штурмі кілька людей збираються разом і зосереджуються на певному питанні. Ідея полягає в тому, щоб забезпечити, щоб дискусія не обмежувалася добрими ідеями або ідеями, які мають безпосередній сенс, а скоріше працювати разом, щоб розкрити якомога більше ідей. Мозковий штурм найкраще працює з модератором, тому що модератор може мотивувати групу і тримати її зосередженою. Мозковий штурм найкраще працює, коли питання є тригерне, на яке потрібно обов’язково відповісти, і кожному надається можливість внести все, що спадає на думку, спочатку на папері. Хорошим основоположним методом для цього процесу, є який називається технікою номінальної групи. Тригерні питання, такі як, яке основні завдання ви виконуєте?,''Які функції ви хотіли б бачити в інструментах?’'або «Які труднощі ви відчуваєте у своїй щоденній роботі?,'' можуть призвести до великих списків цінних ідей, які потім можна обговорити більш детально, ранжувати та проаналізувати.

Фокус-групи схожі на мозковий штурм. Однак фокус-групи виникають, коли групи людей об'єднуються, щоб зосередитися на певному питанні (а не просто генерувати ідеї). Вони також залучають модераторів, щоб зосередити групову дискусію та переконатися, що кожен має можливість взяти участь. Існує великий обсяг письмового матеріалу про те, як правильно розробляти та модерувати фокус-групи.

Переваги.

Мозковий штурм та фокус-групи - це добрі методи збору даних, які можна використовувати, коли людина новачок у галузі та шукає ідеї для подальшого дослідження. Вони добре вміють швидко визначати, що є важливо для учасників. Дві головні побічні переваги мозкового штурму та фокус-груп полягають у тому, що вони можуть познайомити дослідників та учасників один з одним і додатково дати учасникам більше відчуття участі в дослідницькому процесі. Проведення досліджень у польових середовищах часто є стресовим для учасників дослідження; вони, швидше за все, будуть охочими учасниками, якщо вони відчувають себе комфортно з дослідниками і відчувають, що вони є партнерами в дослідженнях, які зосереджені на питаннях, які вони вважають важливими.

Недоліки.

Якщо модератор не дуже добре навчений, мозковий штурм та фокус-групи можуть стати занадто нефокусованими. Хоча техніка номінальної групи допомагає людям висловлювати свої ідеї, люди все ще можуть бути сором'язливими в групі і не говорити те, що вони насправді думають. Те, що учасник піднімає певні питання, це не означає, що проблеми дійсно мають відношення до їхньої щоденної роботи. Часто важко запланувати сесію мозкового штурму або фокус-групу з напруженим графіком інженерів.

Приклади.

Випадок коли використовували мозковий штурм під час першої зустрічі з групою проектування продукту. Метою дослідження було вивчення процесу мобільної співпраці між членами команди дизайнерів. Зустріч з мозковим штурмом була проведена для виявлення проблем та можливих рішень, на яку бачить команда. Ця зустріч дала дослідникам початкове розуміння роботи команди, а також дала дослідникам знати, як існуючі технології або підтримують, або гальмують роботу. Приємним побічним ефектом зустрічі було те, що вона дала дослідникам точку входу для спілкування про процес проектування зі своїми колегами у відділі дизайну Apple.

Інший випадок засновано на великому дослідженні за участю фокус-груп для розуміння вдосконалення процесу програмного забезпечення. У дослідженні використовували 13 компаній-розробників програмного забезпечення та впровадили 49 фокус-груп. Групи складалися від 4 до 6 учасників. Компанії були обрані на основі певних характеристик, але в цілому були репрезентативними для галузі. Кожен сеанс тривав 90 хвилин. Було три типи груп: старші менеджери, менеджери проєктів та розробники. Фокус-групи були модеровані та розглянуті дуже конкретні питання, спрямовані на розуміння кількох факторів, що призводять до успіху та невдачі у вдосконаленні програмного процесу.

Інтерв'ю та анкети

І інтерв'ю, і анкети зосереджені на тому, щоб поставити ряд питань. Питання можуть бути закритими, тобто з множинним вибором, або вони можуть бути відкритими, тобто розмовними відповідями; найкраще завжди мати принаймні деякі відкриті питання, щоб отримати інформацію, яка не може бути передана більш конкретними питаннями пошуку інформації. Щоб ефективно реалізовувати інтерв'ю та анкети, питання та форми повинні бути ретельно розроблені, щоб гарантувати, що зібрані дані мають сенс. Погано сформульоване запитання призводить до неоднозначних відповідей, які неможливо інтерпретувати чи проаналізувати. Настійно рекомендується пілотно протестувати питання або форми, а потім переробити їх, коли ви дізнаєтеся, які питання однозначно впливають на актуальні проблеми.

Для того, щоб отримати хороші статистичні результати з інтерв'ю або анкети, необхідно вибрати вибірку, яка є репрезентативною для учасників, що цікавить. Ця вимога особливо складна в інженерії програмного забезпечення, тому що не вистачає хорошої демографічної інформації про розробників та супроводжуючих. Однак цей недолік не повинен заважати використовувати інтерв'ю та анкети для проведення польових досліджень, якщо не маємо наміру проводити статистичні тести на даних або коли проблема або популяція невелика і чітко визначена. Інтерв'ю та анкети часто проводяться в одній серії досліджень, при цьому інтерв'ю надають додаткову інформацію до відповідей з анкет.

Переваги.

Люди знайомі з відповідями на запитання, або усно, або на папері, і в результаті вони, як правило, їм комфортно з цим методом збору даних. Тому користуються можливістю відповісти на запитання про свою роботу.

Недоліки.

Інтерв'ю та анкети покладаються на самозвіти респондентів про їхню поведінку або ставлення. Ця залежність може спотворити результати кількома способами. Люди не є ідеальними реєстраторами подій навколо них; зокрема, вони переважно повторюють події, які є значущими для них. Наприклад, в одній з анкет учасники повідомили, що читання документації було трудомістким аспектом їхньої роботи, але за 40 годин спостереження не бачили, щоб хтось це робив.

Якщо метою інтерв'ю та анкетування є отримання статистики на основі відповідей на фіксовані запитання, то виникають питання вибірки. Більшість досліджень у галузі інженерії програмного забезпечення повинні використовувати так звану вибірку зручності, що означає, що ми залучаємо всіх, хто доступний, і волонтерів. Це призведе до різних типів упередженості, таких як упередження самовибору (ті, хто найбільше цікавиться нашою роботою, можуть мати інші характеристики від популяції в цілому). Тому про результати завжди потрібно повідомляти з підтвердженням потенційних упереджень та інших загроз обґрунтованості. І результати слід використовувати з урахуванням упереджень. Але, у більшості випадків злегка упереджені дані все ще набагато корисніші, ніж повна відсутність даних.

Інтерв'ю

Особисті інтерв'ю включають принаймні одного дослідника, який особисто розмовляє принаймні з одним респондентом одночасно. Зазвичай фіксований список ретельно сформульованих питань є основою інтерв'ю. Залежно від мети дослідження, респондентів можна заохотити детально вивчити області та трохи відхилитися від сценарію. Телефонні інтерв'ю - це середина між інтерв'ю віч-на-віч та анкетами. У вас є інтерактивність інтерв'ю за ціною та зручністю телефонного дзвінка. Телефонні інтерв'ю не такі особисті, як інтерв'ю віч-на-віч, але вони все ще надають дослідникам можливості для уточнення питань та подальшого дослідження цікавих відповідей. Хоча ця техніка популярна в опитування громадської думки та дослідження ринку, воно мало використовується в емпіричній інженерії програмного забезпечення.

Переваги.

Інтерв'ю дуже інтерактивні. Дослідники можуть уточнити питання для респондентів і дослідити несподівані відповіді. Інтерв'юери також можуть налагодити стосунки з респондентом, щоб покращити якість відповідей.

Недоліки.

Інтерв'ю неефективні за часом і витратами. Контакт з респондентом повинен бути запланований, і принаймні одна людина, як правило, дослідник, повинна поїхати на зустріч (якщо вона не проводиться по телефону, але це зменшує зв'язок, якого можна досягти). Якщо дані інтерв'ю складаються з аудіо- або відеокасет, їх потрібно транскрибувати та/або закодувати; однак ретельне конспектування часто може бути адекватною заміною аудіо- або відеозапису.

Приклади.

Інтерв'ю використовувалися в багатьох дослідженнях, тому що вони добре вписуються в багато типів запитів. Використовували інтерв'ю в поздовжніх дослідженнях як допомогу в розумінні того, як новачки адаптуються до команди розробників та програмної системи. Брали інтерв'ю у новачків раз на три тижні протягом декількох місяців, щоб відстежувати їхній прогрес як членів команди супроводження. Оскільки це було розьяснувальне дослідження, питання були відкритими.

Використовували інтерв'ю для вивчення процесу проектування, який використовується в 19 різних проектах у різних організаціях. Взяли інтерв'ю у персоналу з трьох різних рівнів проєктів-учасників, системних інженерів, старших розробників програмного забезпечення та менеджерів проєктів. Дослідники провели 97 інтерв'ю, які призвели до понад 3000 сторінок стенограм аудіозаписів. Вони виявили три ключові проблеми, загальні для процесів проектування: збої зв'язку та координації, коливання та суміжні вимоги до продукту, а також тенденція до того, що знання прикладної області були локальними в окремих осіб по всій компанії. Вони охарактеризували проблеми на кожному рівні моделі, яку вони згодом визначили.

Використовували інтерв'ю досвідченого персоналу та вищого керівництва, щоб вивчити, як зміни в інженерному процесі вимог вплинули на практику розробки програмного забезпечення. Оскільки історичні дані про попередній процес вимог були обмежені, інтерв'ю були ключовими для надання інформації про те, як зміни впливають на поточну практику. На додаток до початкових інтерв'ю, подальші інтерв'ю були проведені після анкети для з'ясування відповідей. Загалом, виявили, що процес вдосконалення вимог був корисним для команди розробників продукту, що це призвело до кращої документації вимог, кращого розуміння потреб ринку та кращого розуміння вимог. Однак краще спілкування, співпраця та залучення інших груп зацікавлених сторін все ще потрібні.

Анкети

Анкети - це набори питань, які проводяться в письмовому форматі. Це найпоширеніший польовий метод, тому що їх можна вводити швидко і легко. Однак дуже уважно потрібно звернути увагу на формулювання питань, макет форм та впорядкування питань, щоб забезпечити достовірні результати.

Переваги.

Анкети є ефективними за часом і витратами. Дослідникам не потрібно планувати сесії з інженерами, щоб керувати ними. Їх можна заповнити, коли інженер має час між завданнями, наприклад, очікування інформації або під час компіляції. Анкети на основі паперової форми можуть бути доставлені до відповіданого за трохи більше, ніж вартість поштових витрат. Веб-анкети коштують ще дешевше, оскільки паперові форми усуваються, а дані отримуються в електронній формі. Анкети також можуть легко збирати дані від великої кількості респондентів у географічно різноманітних місцях.

Недоліки.

Оскільки інтерв'юера немає, неоднозначні та погано сформульовані питання є проблематичними. Незважаючи на те, що інженерам відносно легко заповнити анкети, вони все одно повинні зробити це самостійно і можуть не знайти часу. Таким чином, норми повернення може бути відносно низьким, що негативно впливає на репрезентативність вибірки. Якщо метою анкети є збір даних для суворого статистичного аналізу з метою спростування гіпотези, то буде потрібно норми набагато вище. Однак, якщо мета полягає в тому, щоб зрозуміти тенденції з розумною впевненістю, то низькі показники відповіді цілком можуть бути в порядку. Однорідність популяції та використовувана техніка відбору також впливають на ступінь, в якій можна узагальнити результати обстежень. На додаток до вищесказаного, відповіді, як правило, більш лаконічні, ніж з інтерв’ю.

Приклади.

Використовував анкети, які були частково веб-, а частково паперовими, щоб дізнатися, які знання інженери програмного забезпечення застосовують у своїй щоденній роботі, і як це порівнюється з тим, чого їх навчали в офіційній освіті. Респондентам було поставлено чотири запитання щодо кожного з довгого списку тем. Провели кілька пілотних досліджень для анкет, але, тим не менш, при аналізі даних виявили, що кілька тем були інтерпретовані різними респондентами по-різному. Незважаючи на це, змогли зробити багато корисних висновків про те, як слід навчати інженерів.

Використав паперову анкету для перевірки дев'яти гіпотез про фактори, що впливають на прийняття інструменту CASE у 52 організаціях Фінляндії. Автор зв'язався з організаціями, які придбали інструменти CASE, та опитав персонал ключових інформаційних систем щодо використання інструменту. Висновки - Компанії та приватні особи частіше використовували інструменти CASE, коли прийняття було добровільним, інструмент сприймався як кращий за свого попередника (попередників), і була підтримка керівництва.

Використовували анкети для отримання інформації від команди продуктів. Всі джерела інформації з дослідження були об'єднані для триангуляції даних і, таким чином, сприяння більшій внутрішній обґрунтованості. Один дослідник використовував веб-анкету для вивчення поведінки вихідного коду. Звернулися до респондентів із семи груп компании заповнити анкету за вказаною веб-адресою. В анкеті використовувався двосторінковий формат. Перша сторінка інформувала учасників про цілі дослідження та їхні права, а на другій сторінці відображалися питання. Використовуючи комбінацію відкритих та закритих питань,визначили набір з одинадцяти пошукових архетипів.

Концептуальне моделювання

Під час концептуального моделювання учасники створюють модель якогось аспекту своєї роботи, метою якої є висвітлення своїх ментальних моделей. У найпростішій формі учасники малюють діаграму якогось аспекту своєї роботи. Наприклад, інженерів-програмістів можуть попросити намалювати діаграму потоку даних, діаграму потоку управління або діаграму упаковки, що показує важливі архітектурні кластери їхньої системи. Як ортогональне використання, розробників можуть попросити намалювати фізичну карту свого оточення, вказуючи, з ким вони розмовляють і як часто.

Більш складна версія концептуального моделювання передбачає використання такого інструменту, як інструмент CASE, або інструмент, спеціально розроблений для концептуального моделювання. Дослідники можуть попросити учасників створити моделі з нуля, або вони можуть створити діаграми, а потім попросити учасників або підтвердити їх, або запропонувати модифікації.

Переваги.

Системні ілюстрації забезпечують точне зображення уявлення користувача про його ментальну модель системи. Такі моделі легко зібрати і вимагають лише низькотехнологічних засобів (ручки та паперу).

Недоліки.

Результати концептуального моделювання важко інтерпретувати, особливо якщо дослідник не має знань про систему. Деякі інженери неохоче малюють, а якість та рівень деталей на діаграмах можуть значно відрізнятися.

Приклади.

Описує початкові зусилля з формального моделювання процесів розробки в проекті розробки з відкритим кодом. Описує повну модель зображення одного типу проекту з відкритим кодом, який пов'язує різні ролі, артефакти та інструменти з процесом розробки в цілому. Крім того, описує формальну модель, яка була побудована за допомогою мови моделювання процесів. Модель була побудована з використанням мета-моделі процесу як її семантичної основи. Метою моделювання було забезпечити формальне розуміння, яке можна було б проаналізувати, поділитися та порівняти в дослідницьких та практичних спільнотах.

У груповому дослідженні інструкції учасникам дослідження полягали в тому, щоб зрозуміти систему. Ці інструкції виявилися занадто розпливчастими. Деякі учасники намалювали діаграми потоку даних, деякі малювали архітектурні кластери, інші перерахували важливі структури даних та змінні тощо. Не дивно, що менеджер групи згодом зазначив, що системні ілюстрації відображали поточні проблеми, над якими працювали різні інженери-програмісти.

Робочі щоденники Work Diaries

Робочі щоденники вимагають від респондентів записувати різні події, які відбуваються протягом дня. Це може включати заповнення форми в кінці дня, запис конкретних дій, коли вони відбуваються, або відмітку будь-якого поточного завдання в заздалегідь вибраний час. Ці щоденники можна зберігати на папері або в комп'ютері. Паперові форми адекватні для запису інформації в кінці дня. Комп'ютерна програма може бути використана для того, щоб спонукати користувачів до введення у випадковий час.

Переваги.

Робочі щоденники можуть надавати кращі самозвіти про події, оскільки вони записують діяльність на постійній основі, а не в ретроспективі. Випадкова вибірка подій дає дослідникам можливість зрозуміти, як інженери проводять свій день, не здійснюючи багато спостережень або затінення.

Недоліки.

Робочі щоденники все ще покладаються на самозвіти; зокрема, ті, які вимагають від учасників згадувати події, можуть мати значні проблеми з точністю. Ще одна проблема з робочими щоденниками полягає в тому, що вони можуть заважати респондентам під час роботи. Наприклад, якщо інженерам доводиться записувати кожен раз, коли вони йдуть і консультуються з колегою, вони можуть консультуватися рідше. Вони також можуть забути або нехтувати записом деяких подій і можуть не записувати на очікуваному рівні деталізації.

Приклади.

Співпраця у великій компанії з розробки програмного забезпечення. На додаток до спостережень та інтерв'ю, попросили інженерів записати свої схеми спілкування протягом одного дня. Дослідники були зацікавлені як у взаємодії між членами команди, так і в типових моделях спілкування розробників. Вони виявили, що розробники часто і широко спілкуються, і використовують багато різних типів методів спілкування, перемикаючись між ними відповідно. Вони також виявили, що моделі спілкування сильно різняться між окремими розробниками.

Інший приклад. Випадковим чином вибрано супроводжуючих програмного забезпечення і попросили їх заповнити форму, щоб описати їх наступне завдання. Ці звіти були використані для профілю розподілу частоти завдань супроводження. Тридцять три гіпотези були перевірені, і деякі з них були підтримані. Наприклад, продуктивність програміста (рядки коду за одиницю часу) прогнозується розміром завдання, типом змін, але вона не прогнозується досвідом супроводжуючого, віком програми або розміром програми.

Протоколи Think-Aloud

В аналізі протоколу Think-Aloud дослідники просять учасників думати вголос під час виконання завдання. Завдання може відбуватися природним чином на роботі або бути заздалегідь визначеним дослідником. Оскільки інженери іноді забувають вербалізувати, експериментатори можуть іноді нагадувати їм продовжувати думати вголос. Окрім цих випадкових нагадувань, дослідники не втручаються у процес вирішення проблем. Сеанси Think-loud зазвичай тривають не більше двох годин. Аналіз протоколу Think-aloud найчастіше використовується для визначення або перевірки когнітивної моделі, оскільки інженери виконують деякі завдання з програмування.

Переваги.

Просити людей думати вголос відносно легко реалізувати. Ця техніка дає унікальний погляд на процес вирішення проблем і додатково дає доступ до ментальної моделі.

Недоліки.

Аналіз протоколу Think aloud був розроблений для використання в ситуаціях, коли дослідник міг відобразити весь проблемний простір. Незрозуміло, як цей метод перекладається на інші домени, де неможливо апріорі знати, що таке проблемний простір. Навіть за допомогою ручного ведення обліку, аналізувати дані мислення вголос важко і трудомістко.

Приклади.

Попросили розробників програмного забезпечення думати вголос, оскільки вони виконували завдання з супроводження, яке вимагало розуміння програми. Обидва інженери, які брали участь в експерименті, обрали сеанси налагодження. Протоколи think-aloud були закодовані, щоб визначити, чи дотримувалися учасники інтегрованої мета-моделі розуміння програми, визначеного цими дослідниками. Вони знайшли докази використання цієї моделі, і тому змогли використовувати модель, щоб запропонувати вимоги до інструментів для середовищ супроводження програмного забезпечення.

Були зацікавлені в оцінці інтерфейсу користувача для прототипу системи управління.Попросили кількох суб'єктів вибрати з набору визначених проблем, а потім вирішити проблему за допомогою системи. Суб'єктів попросили вербалізувати свої думки та мотивацію під час роботи над проблемами. Дослідники змогли визначити позитивні та негативні аспекти інтерфейсу користувача та використовувати цю інформацію в еволюції системи.

Затінення/спостереження Shadowing/Observation

У затіненні експериментатор слідує за учасником і записує його діяльність. Затінення може відбуватися протягом необмеженого періоду часу, якщо є охочий учасник. Тісно пов'язане з затіненням, спостереження відбувається, коли експериментатор спостерігає за інженерами, які беруть участь у їхній роботі, або конкретними завданнями, пов'язаними з експериментами, такими як зустрічі або програмування. Різниця між затіненням та спостереженням полягає в тому, що дослідник затіняє одного інженера одночасно, але може спостерігати за багатьма.

Переваги.

Не вимагає спеціального обладнання. Затінення та спостереження легко реалізувати, дають швидкі результати.

Недоліки.

Для затінення часто важко зрозуміти, що робить інженер, особливо коли вони використовують комбінації клавіш для видачі команд і працюють швидко. Однак для загальної картини, наприклад, знаючи, що вони зараз налагоджують, затінення працює добре. Спостерігачі повинні мати досить добре розуміння навколишнього середовища, щоб інтерпретувати поведінку інженера. Іноді це можна компенсувати попереднім визначенням набору категорій або шуканої поведінки. Звичайно, знову ж таки, це обмежує тип даних, які будуть зібрані.

Приклади.

Впровадили затінення в роботу двома способами. По-перше, один експериментатор зробив нотатки з паперу та олівця, щоб вказати, що робив учасник і приблизно як довго. Ця інформація дала гарну загальну картину робочих звичок інженерів. Також розробили метод, який називаємо синхронізованим затіненням. Тут використовували двох експериментаторів і замість олівця та паперу використовували два ноутбуки для запису дій інженера. Один відповідав за встановлення цілей високого рівня учасників, тоді як інший відповідав за запис їхніх дій низького рівня. Використовували заздалегідь визначені категорії (макроси Microsoft Word), щоб полегшити запис.

Як приклад спостереження, цікавилися тим, чи впливає спільне розміщення членів команди на розробку програмного забезпечення. На додаток до інтерв'ю та анкет, спостерігали за командами, конференц-дзвінками, вирішенням проблем та фотографували різні артефакти. Дослідники виявили, що задоволення та продуктивність зросли для команд, які розміщуються в спільному місті.

Спостереження за учасником (приєднання до команди) Participant Observation (Joining the Team)

У методі учасника-спостерігача дослідник, по суті, стає частиною команди та бере участь у ключових заходах. Участь у процесі розробки програмного забезпечення забезпечує досліднику високий рівень знайомства з членами команди та завданнями, які вони виконують. В результаті інженери почуваються комфортно з присутністю дослідника і, як правило, не помічають, що їх спостерігають.

Переваги.

Респонденти, швидше за все, почуваються комфортно з членом команди і діють природно під час спостереження. Дослідники також розвивають більш глибоке розуміння завдань після виконання їх у контексті групи інженерії.

Недоліки.

Приєднання до команди займає багато часу. Потрібно значну кількість часу, щоб встановити справжнє членство в команді. Крім того, дослідник, який стає занадто залученим, може втратити перспективу на явище, яке спостерігається.

Приклади.

Учасник-спостерігач використовуються у дослідженнях того, як комунікаційні та організаційні фактори впливають на якість перевірок програмного забезпечення). Один спостерігач був інтегрований у новостворену команду розробників. За сімнадцять місяців спостерігач взяв участь у двадцяти трьох інспекційних зустрічах. Завдяки участі розробили ряд гіпотез про те, як такі фактори, як знайомство, організаційна відстань та фізична відстань, пов'язані з тим, скільки часу витрачається на обговорення та завдання.

Використовували метод учасник-спостерігач коли один з дослідників, докторант, приєднався до досліджуваної команди розробників як засіб відстеження прогресу експерименту, збору та перевірки даних та спостереження за інспекціями. Тут техніка польових досліджень була використана на службі більш традиційних експериментальних методів.

Методи другого ступеня: непряме залучення інженерів-програмістів

Методи другого ступеня вимагають від дослідника доступу до середовища інженера-програміста. Однак методи не вимагають прямого контакту між учасником і дослідником. Спочатку ініціюється збір даних, потім інженери приходять до своєї звичайної роботи, і, нарешті, дослідники повертаються, щоб обрати дані, які були автоматично зібрані. В результаті ці методи вимагають дуже мало або взагалі не вимагають часу від інженерів і підходять для довготривалих досліджень.

Інструментальні системи

Ця техніка вимагає, щоб інструмент, який збирає дані був вбудоване в програмні інструменти, які використовуються інженером. Цей інструмент використовується для автоматичного запису інформації про використання програмних інструментів. Інструмент можна використовувати для моніторингу того, як часто використовується програмний інструмент або функція, шаблони доступу до файлів і каталогів, і навіть терміни, що лежать в основі різних дій. Цю техніку також називають системним моніторингом. У деяких випадках інструменти просто записують команди, видані користувачами. Більш просунуті форми дуже детально записують як вхід, так і вихід, щоб дослідник міг ефективно відтворити сесію.

Переваги

Моніторинг системи не вимагає часових зобов'язань від інженерів. Оскільки люди, як правило, дуже погано оцінюють такі фактори, як відносна частота та тривалість різних видів діяльності, які вони виконують, цей метод можна використовувати для точного надання такої інформації.

Недоліки.

Важко осмислено аналізувати дані з інструментальних систем; тобто важко визначити думки та цілі інженерів. Інструментарій може сказати, що робив інженер, але вони не викривають мислення, що стоїть за цією дією. Ця проблема особливо актуальна, коли робоче середовище погано зрозуміле або обмежене. Наприклад, інженери часто налаштовують свої середовища, додаючи скрипти та макроси. Один із способів впоратися з цим недоліком - відтворити події інженеру і попросити його прокоментувати. Хоча в багатьох юрисдикціях роботодавці мають право контролювати працівників, існують етичні проблеми, якщо дослідники беруть участь у моніторингу інженерів без їхнього відома.

Приклади.

Використовували інструмент реєстрації оцінці того, наскільки корисним був конкретний інструмент CASE. Реєстратор записував дані щоразу, коли відбувалася подія. Події були заздалегідь визначені раніше. Текстові дані не були записані. Дослідники виявили, що запис подій був лише недоліком їхнього дизайну. Було б доречніше зібрати інформацію про контекст конкретної події. Використовував VNC (Virtual Network Computing) для збору дослівних екранних протоколів (безперервних скріншотів) розробників програмного забезпечення, які займаються розробкою програмного забезпечення. Також збирав словесні протоколи та використовував теоретичний підхід для аналізу даних.

Fly on the Wall (Учасники записують власну роботу)

Це гібридна техніка. Це дозволяє досліднику бути спостерігачем діяльності, не будучи присутнім. Учасників просять самостійно записати відео- або аудіозапис, коли вони займаються якоюсь заздалегідь визначеною діяльністю.

Переваги.

Метод вимагає від учасників дуже мало часу і дуже ненав'язливий. Хоча на початку може бути певний дискомфорт, це швидко зникає.

Недоліки.

Учасники можуть забути увімкнути звукозаписне обладнання у відповідний час, і в результаті запис може бути неповним або відсутнім. Камера фіксована, тому контекст того, що записано, може бути важко зрозуміти. Аналіз отриманих даних коштує високу вартість.

Приклади.

Попросив наставників та учнів програмної організації зробити аудіозапис своїх зустрічей, щоб вивчити, як передається досвід. Пізніше проаналізували ці записи на наявність закономірностей у розмовах. Виявили, що дискусії мали дуже інтерактивний характер, використовуючи такі методи, як підтвердження та повторне повідомлення для перевірки повідомлень. Наставники не тільки пояснюють особливості системи; вони також надають обґрунтування дизайну. Хоча наставництво є ефективним, воно також займає багато часу, тому робить деякі пропозиції щодо документації та пропонує короткі курси для учнів.

Мали відеозйомки зустрічей команди інженерів під час етапу проектування проєкту розробки. Дослідники не брали участі в зустрічах, і ці стрічки служили первинними даними для дослідження. Метою дослідження було зрозуміти, як командна робота, цілі та дизайн розвивалися протягом чотирьох місяців. Спочатку команда зосередилася на зборі знань про область застосування, потім на вимогах до програми і, нарешті, на дизайнерських підходах. Дослідники також виявили, що команда не змогла записати більшу частину ключової інформації; в результаті вони переглянули питання, які були врегульовані на попередніх зустрічах.

Вивчали моделі взаємодії між інженерами на зустрічах з технічного огляду. Інженерам просто потрібно було вмикати відеореєстратор щоразу, коли вони проводили зустріч. Дослідники проаналізували сценарії сесій і змоделювали типи взаємодій, які відбувалися під час зустрічей. Їх аналіз призвів до рекомендацій щодо способів вдосконалення таких зустрічей.

Методи третього ступеня (Аналіз робочих артефактів):

Методи третього ступеня намагаються розкрити інформацію про те, як працюють інженери, дивлячись на їх результати та побічні продукти. Прикладами їх результатів є вихідний код, документація та звіти. Побічні продукти створюються в процесі виконання роботи, наприклад, робочі запити, журнали змін та вихід з інструментів управління конфігурацією та збірки. Ці сховища або архіви можуть служити основним джерелом інформації. Іноді дослідники наймають інженерів для допомоги в інтерпретації або перевірці даних. Є деякі переваги та недоліки, які є загальними для всіх методів третього ступеня.

Перевага.

Методи третього ступеня майже не вимагають жодних часових зобов'язань від інженерів.

Недолік.

Зібрані дані дещо віддалені від фактичного процесу розробки; Старі дані можуть стосуватися систем або процесів, які з тих пір були значно змінені, а останні дані відображають лише дуже вузький погляд на розробку програмного забезпечення. Крім того, може бути важко інтерпретувати дані змістовно. Документацію може бути важко зрозуміти, а журнали, як правило, містять загадкові скорочення та коментарі, які, як очікувалося, лише сформують послідовну картину в свідомості інженерів, які спочатку писали та читали їх. У зв'язку з цим, методи третього ступеня, як правило, повинні бути доповнені іншими методами для досягнення цілей дослідження.

Аналіз електронних баз даних виконаної роботи

У більшості великих організацій з розробки програмного забезпечення робота, що виконується розробниками, ретельно керується за допомогою звітності про проблеми, запитів на зміни та систем управління конфігурацією. Рясні записи, які зазвичай залишають такі системи, є багатим джерелом інформації для дослідників. Деяка інформація записується автоматично, наприклад, щоразу, коли модулі реєструються в бібліотеці та виводяться з неї; решта інформації - це описи та коментарі, введені вручну інженерами.

Переваги.

Велика кількість даних часто легко доступна. Дані стабільні і на них не впливає присутність дослідників.

Недоліки.

Може бути невеликий контроль над кількістю та якістю інформації, введеної вручну про виконану роботу. Наприклад, виявили, що описові поля часто не заповнюються або заповнюються різними розробниками по-різному. Також важко зібрати додаткову інформацію про запис, особливо якщо він дуже старий або інженер, який працював над ним, більше не доступний.

Приклади.

Робочі записи можна використовувати кількома способами. Проаналізували повідомлення про несправності в системі управління повітряним рухом, щоб оцінити ефект додавання формальних методів до процесу розробки. Кожен модуль у програмній системі був розроблений з використанням одного з трьох формальних методів або неформального методу. Хоча код, розроблений з використанням формальних методів, як правило, мав менше помилок, результати не були переконливими навіть у поєднанні з іншими даними аудиту коду та модульного тестування.

Протестували модель взаємозв'язків між типами супроводження та атрибутами модулів, що змінюються. Їх аналіз базувався на історії змін, яка містила інформацію про дату створення та автора, функцію модуля, інженера, який вніс зміни, та опис зміни. Вони знайшли підтримку своїм регресійним моделям і припущення, що діяльність з супроводження програмного забезпечення слідує передбачуваним шаблонам. Наприклад, модулі, які є більш складними, відносно великими та старими, потрібно відновлювати частіше, тоді як модулі з важливою функціональністю, як правило, вдосконалюються частіше.

Дослідники НАСА вивчали дані з різних проектів у своїх дослідженнях, щоб ефективно використовувати COTS. Вони розробили великий звіт, який рекомендує, як покращити процеси, які використовують COTS.

Аналіз журналів інструментів

Багато програмних систем, що використовуються інженерами, генерують журнали в тій чи іншій формі. Наприклад, інструменти автоматичного будівництва часто залишають записи, як і сервери ліцензій, які контролюють використання інструментів CASE. Деякі організації вбудовують складну реєстрацію в широкий спектр інструментів, щоб вони могли краще зрозуміти потреби інженерів у підтримці. Такі журнали інструментів можуть бути проаналізовані так само, як інструменти, які були вбудовані дослідниками. Різниця полягає лише в тому, що для цієї техніки третього ступеня дослідники не мають контролю над видом зібраної інформації. Ця техніка також схожа на аналіз баз даних виконаної роботи, за винятком того, що остання включає дані, введені вручну інженерами. Аналіз журналів інструментів став дуже популярною областю досліджень.

Переваги.

Дані вже в електронній формі, що полегшує кодування та аналіз. Поведінка, що реєструється, є частиною звичайної роботи інженерів.

Недолік.

Компанії, як правило, використовують різні інструменти по-різному, тому важко послідовно збирати дані при використанні цієї техніки з кількох організацій.

Приклади.

Аналізували файли журналів, створені інструментами збірки. Розробили інструменти для автоматичного вилучення інформації з відповідних подій з цих файлів. Ці дані були введені в реляційну базу даних разом з інформацією, зібраною з інших джерел.

Переглянули журнали використання інструментів, зібрані групою інструментів, щоб визначити, які інструменти інженери по всій компанії (на відміну від групи, яку вивчали) використовували найбільше. Виявили, що пошук та інструменти Unix використовувалися особливо часто.

Використовували дані, створені системою управління змінами, щоб краще зрозуміти, як відбувається спілкування в глобально розподіленій розробці програмного забезпечення. Використовували кілька методів моделювання, щоб зрозуміти взаємозв'язок між інтервалом запиту на модифікацію та іншими змінними, включаючи кількість залучених людей, розмір зміни та розподілений характер груп, що працюють над зміною.

Використовували дані опитування для з'ясування та підтвердження висновків з аналізу журналів інструментів. Загалом, виявили, що розподілена робота вводить затримку. Пропонують деякі механізми, які впливають на цю затримку, в першу чергу те, що розподілена робота включає більше людей, що робить запити на зміни довшими для завершення.

Аналіз документації

Ця техніку зосереджено на документації, створеній інженерами, включаючи коментарі в програмному коді, а також окремі документи, що описують програмну систему. Дані, зібрані з цих джерел, також можуть бути використані для реінжинірії, такої як ідентифікація підсистеми. Інші джерела документації, які можна проаналізувати, включають локальні групи новин, групові списки електронної пошти, нотатки та документи, які визначають процес розробки.

Переваги.

Документи, написані про систему, часто містять концептуальну інформацію і представляють уявлення про розуміння програмної системи принаймні однією людиною. Вони також можуть служити вступом до програмного забезпечення та команди. Коментарі в програмному коді, як правило, надають низькорівневу інформацію про алгоритми та дані. Використання вихідного коду як джерела даних щодо інтерфейсу програмної системи.

Недоліки.

Вивчення документації може зайняти багато часу і вимагає певних знань про джерело. Письмові матеріали та коментарі джерела можуть бути неточними.

Приклади.

Конференції ACM SIGDOC містять багато досліджень документації. У проекті концептуального моделювання, створили початковий список концепцій частково шляхом обробки документації. Результат було використано для створення системи дослідження коду, в якій технічні терміни стали гіперпосиланнями. Це дозволило людям знайти подібний код та відповідну документацію.

Статичний та динамічний аналіз системи

У цьому методі аналізується код (статичний аналіз) або сліди, створені при запуску коду (динамічний аналіз), щоб дізнатися про дизайн, і опосередковано про те, як думають і працюють інженери. Можна порівняти програмування або архітектурні стилі кількох інженерів, аналізуючи їх використання різних конструкцій або значення різних показників складності.

Переваги.

Вихідний код, як правило, легко доступний і містить дуже велику кількість інформації, готової до видобутку.

Недоліки.

Для вилучення корисної інформації з вихідного коду потрібні парсери та інші інструменти аналізу; виявили, що така технологія не завжди є зрілою. Хоча парсери, що використовуються в компіляторах, мають високу якість, парсери, необхідні для певних видів аналізу, можуть бути зовсім іншими, наприклад, їм, як правило, потрібно аналізувати код без його попередньої обробки. Аналіз старих систем, створених кількома програмістами протягом багатьох років, може ускладнити розділення різних незалежних змінних (програмістів, діяльності тощо), які породжують різні стилі, метрики тощо.

Приклади.

- Використовують методи статичного аналізу, що включають відповідність шаблонів, щоб виявити шаблони дизайну у вихідному коді. Вони вказують на те, що саме ці моделі мислення знаходяться в основі багатьох ключових елементів великомасштабних програмних систем, і що для того, щоб зрозуміти ці системи, потрібно відновити і зрозуміти шаблони, на яких вони були побудовані.
- Були зацікавлені в цінності, доданої парним програмуванням над індивідуальним програмуванням. В якості одного з заходів у своєму експерименті вони розглянули кількість тестових випадків, пройдених парами проти окремих програмістів. Вони виявили, що пари генерували код вищої якості, як доказ значно більшої кількості пройдених тестових випадків.
- Використовуємо як статичний, так і динамічний аналіз, щоб виявити різні способи, за допомогою яких систему можна легше зрозуміти. Аналізуємо вихідний код для створення кластерів, які можуть допомогти супроводжуючому візуалізувати погано структуровану систему.

Щоб провести хороші польові дослідження,

- дослідник повинен спочатку створити ефективні плани. Плани повинні описувати методи дослідження, а також те, як слід вирішувати різні практичні питання.
- Щоб вибрати методи дослідження, застосовуємо модифікацію методології GQM, спочатку розроблену для вибору метрик, але описану для вибору методів збору даних. Дослідник повинен мати на увазі цілі, вибрати навчальні питання, які допоможуть досягти цілей, а потім вибрати один або кілька методів, які найкраще підходять для відповіді на питання.
- На додаток до прийняття рішення, які методи використовувати, дослідник також повинен визначити рівень деталізації даних, які будуть зібрані. Для більшості методів першого ступеня, як правило, потрібно вибирати серед них, у порядку збільшення обсягу інформації і, отже, складності аналізу: ручні нотатки, аудіозапис та відеозйомка. У всіх трьох випадках ключовою складністю є кодування даних, щоб їх можна було проаналізувати.

- Незалежно від підходу до збору та аналізу даних, польові дослідження також викликають багато логістичних проблем, які слід вирішити в початковому плані. Наприклад: Як підходити та встановлювати відносини з компаніями та працівниками, щоб отримати відповідну вибірку учасників? Чи буде дослідження вважатися етичним, враховуючи, що в ньому беруть участь людські учасники? І, нарешті, чи можна буде знайти компетентний та зацікавлений науковий персонал, враховуючи, що більшість методів є трудомісткими, але ще не є частиною основних досліджень.